

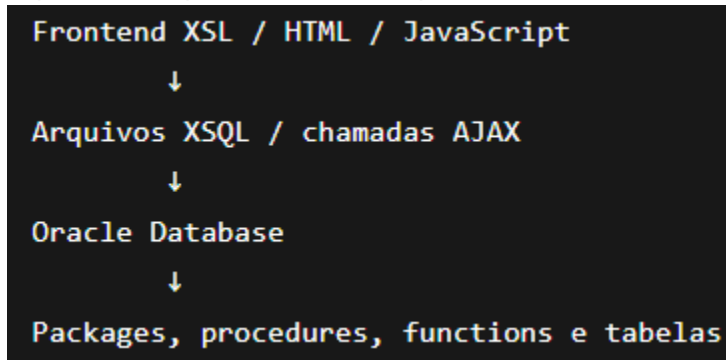


Passagem técnica Do Projeto Stem Market

O objetivo desta apresentação é passar uma visão técnica e operacional do projeto, destacando os principais pontos de manutenção, arquitetura, versionamento, deploy, banco de dados, logs e arquivos críticos.

Introdução

1. O HEALTH é um sistema legado, com forte dependência do banco de dados.
2. Diferente de uma aplicação moderna em camadas, onde temos um backend Java/Spring expondo APIs para o frontend, aqui o funcionamento é mais próximo de um modelo:



- 3.
4. Ou seja, o verdadeiro backend é o banco de dados.

5. Os principais componentes técnicos do projeto são:

```
Servidor: Apache Tomcat legado
Frontend: XSL, HTML, JavaScript, jQuery
Backend lógico: Oracle Database
Camada intermediária: XSQL
Banco: Oracle, com forte uso de packages, procedures e functions
Versionamento: Git
Deploy: via pipeline/servidor, com publicação de arquivos no ambiente
Encoding predominante: ISO-8859-1
```

- 6.
7. Organização do código e Git
8. Repositório Git: [Sign in · GitLab](#)
9. Branch principal: develop e main.
 - a. Branches de trabalho ficam organizadas em issues
 - b. Ambientes: dev e prod
 - c. As tag's respeitamos essa mesma nomenclatura, apenas atribuímos a ela qual é a branch desejada para a implantação.



Banco de dados como backend

No HEALTH, não adianta olhar apenas o JavaScript ou o XSL. Quase todas as respostas estão no banco, ou em alguma configuração, o caminho que eu aprendi a seguir para investigar qualquer problema, normalmente é:

1. Identificar tela/funcionalidade.
2. Identificar arquivo XSL/JS envolvido.
3. Identificar XSQL chamado.
4. Identificar query/procedure/package usado.
5. Validar dados no Oracle.
6. Conferir logs e erros.

Tabelas úteis para suporte e diagnóstico

```
select * from LOGEDU order by Fecha desc;      -- tabela de logs
select * from ERRORES order by err_fecha desc; -- tabela de erros
select * from PORTALESMVM;                     -- tabela de apontamentos/portais
select * from MENSAJES;                       -- tabela de mensagens
select * from MENUS;                          -- tabela de menus
select * from PARAMETROS;                     -- tabela de parâmetros
```

LOGEDU: Tabela usada para logs da aplicação.

É útil para verificar eventos recentes, execuções e rastros técnicos.



ERRORES: Tabela de erros.

É uma das primeiras tabelas para consultar quando o usuário relata falha sistêmica.

PORTALESVM: Tabela relacionada a apontamentos/portais.

Pode ajudar em validações de configuração, ambiente ou vínculo de portal.

MENSAJES: Tabela de mensagens do sistema.

Pode conter textos, mensagens de tela, mensagens internas ou parametrizações relacionadas.

MENUS: Tabela de menus.

Importante quando a demanda envolve liberação, bloqueio ou visualização de funcionalidades.

PARAMETROS: Tabela de parâmetros gerais.

Muito importante porque várias regras do sistema podem ser ativadas/desativadas ou modificadas por parâmetro, sem alteração direta de código.

Pontos que eu deixo registrado como alerta

1. O projeto é legado e acoplado.
2. O banco é parte central da aplicação.
3. Alterações pequenas podem ter impacto grande.
4. Muitos comportamentos dependem de parâmetros no banco.
5. O frontend depende muito de IDs e estrutura de DOM.
6. O JavaScript possui muitas variáveis globais.
7. O encoding precisa ser preservado.
8. O fluxo de licitação é sensível e deve ser testado de ponta a ponta.
9. Nunca assumir que o problema está só no frontend.
10. Sempre validar XSL + XSQL + banco + JS juntos.

PS

De forma geral, o principal ponto que eu gostaria de deixar claro é que o HEALTH não deve ser analisado como uma aplicação moderna tradicional. Ele é um sistema legado, muito apoiado no banco de dados, com telas construídas em XSL, regras distribuídas entre JavaScript, XSQL e Oracle, e bastante dependência de parâmetros e estruturas já existentes.

Então, para dar manutenção com segurança, o ideal é sempre investigar o fluxo completo: tela, JavaScript, chamada XSQL, banco, logs e parâmetros.



Minha recomendação para o próximo time é trabalhar sempre com branch específica, commits pequenos, validação em ambiente controlado, consulta às tabelas de log e erro, e muita atenção aos arquivos de licitação, principalmente quando envolver seleção de ofertas, melhores preços, fornecedores, motivos e geração de pedidos.